

CS-468

Software Quality Assurance & Testing

Week 10

Session 1

Structure based (White box) testing

- Structure-based techniques serve two purposes:
 - test coverage measurement
 - and structural test case design.
- They are often used first to assess the amount of testing performed by tests derived from specification-based techniques, i.e. to assess coverage.
- They are then used to design additional tests with the aim of increasing the test coverage.
- Structure-based test design techniques are a good way of generating additional test cases that are different from existing tests.
- They can help ensure more breadth of testing, in the sense that test cases that achieve 100% coverage in any measure will be exercising all parts of the software from the point of view of the items being covered.

Test Coverage

- Test coverage measures in some specific way the amount of testing performed by a set of tests (derived in some other way, e.g. using specification-based techniques).
- Wherever we can count things and can tell whether or not each of those things has been tested by some test, then we can measure coverage.
- The basic coverage measure is :

$$\text{Coverage} = \frac{\text{Number of coverage items exercised}}{\text{Total number of coverage items}} \times 100\%$$

- where the 'coverage item' is whatever we have been able to count and see whether a test has exercised or used this item.

... Test Coverage

- There is danger in using a coverage measure.
- 100% coverage does *not* mean 100% tested!
- Coverage techniques measure only one dimension of a multidimensional concept.
- Two different test cases may achieve exactly the same coverage but the input data of one may find an error that the input data of the other doesn't.
- One drawback of code coverage measurement is that it measures coverage of what *has* been written, i.e. the code itself; it cannot say anything about the software that has *not* been written.
- If a specified function has not been implemented, specification-based testing techniques will reveal this.
- If a function was omitted from the specification, then experience-based techniques may find it.
- But structure-based techniques can only look at a structure which is already there.

Statement coverage and statement testing

- Statement coverage is calculated by:

$$\text{Statement coverage} = \frac{\text{Number of statements exercised}}{\text{Total number of statements}} \times 100\%$$

Statement coverage and statement testing – Example 1

- Consider code:

```
READ A
READ B
IF A > B THEN C = 0
ENDIF
```

- To achieve 100% statement coverage of this code segment just one test case is required, one which ensures that variable A contains a value that is greater than the value of variable B, for example, A = 12 and B = 10.
- Note that here we are doing structural test design first, since we are choosing our input values in order ensure statement coverage.

Statement coverage and statement testing – Example 2

- Consider code:

```
1  READ A
2  READ B
3  C =A + 2*B
4  IF C> 50 THEN
5      PRINT large C
6  ENDIF
```

Value of A	Value of B	Value of C	Line no. of Statements Covered	No. of statements exercised	Statement coverage (%)
2	3	8	1, 2, 3, 4 & 6	5	83
0	25	50	1, 2, 3, 4 & 6	5	83
47	1	49	1, 2, 3, 4 & 6	5	83
20	25	70	1, 2, 3, 4, 5 & 6	6	100

Decision coverage and decision testing

- A decision is an IF statement, a loop control statement (e.g. DO-WHILE or REPEAT-UNTIL), or a CASE statement, where there are two or more possible exits or outcomes from the statement.
- With an IF statement, the exit can either be TRUE or FALSE, depending on the value of the logical condition that comes after IF.
- With a loop control statement, the outcome is either to perform the code within the loop or not - again a True or False exit. Decision coverage is calculated by:

$$\text{Decision coverage} = \frac{\text{Number of decision outcomes exercised}}{\text{Total number of decision outcomes}} \times 100\%$$

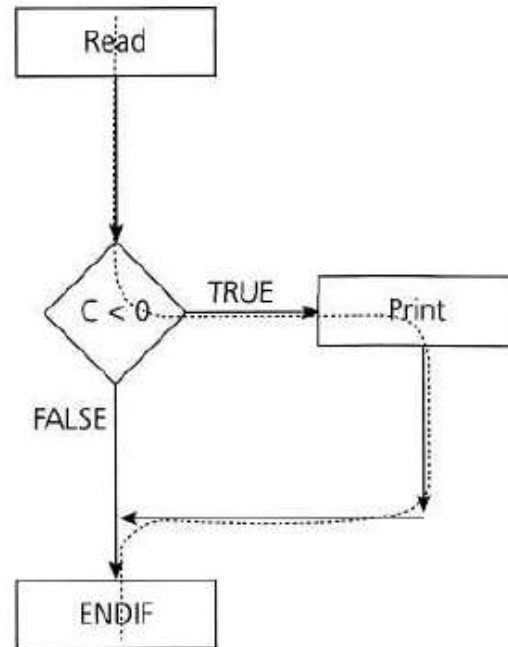
Decision coverage and decision testing – Example 1

- Consider code:

```
1 READ A
2 READ B
3 C = A - 2 * B
4 IF C < 0 THEN
5     PRINT "C negative"
6 ENDIF
```

- Let's suppose that we already have the following test, which gives us 100% statement coverage for code sample

– **A = 20, B = 15**



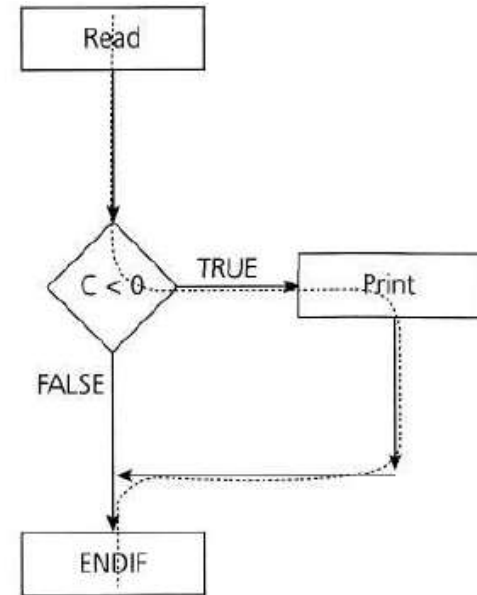
Control flow diagram for code sample

...Decision coverage and decision testing – Example 1

- Which decision outcomes have we exercised with our test?
- The value of C is -10, so the condition ' $C < 0$ ' is True, so we will print 'C negative' and we have exercised the True outcome from that decision statement.
- But we have not exercised the decision outcome of False.
- What other test would we need to exercise the False outcome and to achieve 100% decision coverage?

...Decision coverage and decision testing – Example 1

- Before we answer that question, let's have a look at another way to represent this code. Sometimes the decision structure is easier to see in a control flow diagram.
- The dotted line clearly shows that we haven't yet had a test that takes the False exit from the IF statement.
- Let's modify our existing test set by adding another test:
 1. **A = 20, B = 15**
 2. **A = 10, B = 2**
- This now covers both of the decision outcomes, True and False.
- If we were to draw the path taken by Test 2, it would be a straight line from the read statement down the False exit and through the ENDIF.
- Note that we could have chosen other numbers to achieve either the True or False outcomes.



Control flow diagram for code sample ...

...Sample exam questions

1. How many test cases are needed to achieve 100 per cent decision coverage?
2. How many test cases would be needed for 100 per cent statement coverage?
3. What level of decision/statement coverage would be achieved by the single input $A = 10, B = 2$? → Formula
4. Generate test data for maximum statement coverage
5. Generate test data for maximum decision coverage